

Guion — Demo Harness (15 min)

Texto para decir en voz alta o parafrasear. Marcado con tiempos orientativos y con [EN VIVO: ...] donde corresponde correr algo en la terminal. No hace falta leerlo textual — es para tener el hilo, no para recitarlo.

1. Apertura — qué es Harness para mí (~1 min)

"Cuando nos pidieron traer nuestra propia interpretación de qué es un Harness, la consigna era justamente esa: no hay una definición única, puede ser una skill, un runtime, una plataforma, una combinación de herramientas. Mi interpretación es la más literal: **un harness es el sistema de control automático alrededor de un agente de coding.** Le das specs *antes* de que escriba código — eso es feedforward — y le ponés gates automáticos que verifican *después* lo que produjo — eso es feedback. La idea de fondo es que vos no revisás cada línea que escribe el agente: diseñás las condiciones para que lo que produzca sea correcto por construcción."

2. Por qué minimalista (~1 min)

"Decidí a propósito no irme por las ramas. Podría haber armado algo con múltiples repos, un caso Brownfield, integraciones — pero el objetivo de esta semana no es tener el producto terminado, es validar el concepto. Así que tomé el caso más chico que me permitiera mostrar el ciclo completo: specs → agente que genera código → gate que lo verifica → gate que caza una violación. Un CRUD chico de 'Products' en .NET, Greenfield, para poder demostrar y validar de punta a punta en el tiempo que tenía. Prefiero mostrar el mecanismo completo en algo chico, a mostrar algo grande a medio hacer."

3. Recorrido del proyecto (~3 min) — mientras mostrás las carpetas

specs/ -> el QUÉ gates/ -> el control automático (el corazón del harness) scripts/ -> el loop de feedback src/ -> vacío. Todavía no existe la app.

"Esto es Spec-Driven Development aplicado directo, no de fondo: la spec es el artefacto principal, el código es su expresión."

specs/constitution.md

"Son las reglas globales, no negociables por feature: arquitectura en capas obligatoria — Api depende de Application, Application depende de Domain, Infrastructure solo la toca Application —, prohibiciones explícitas como que los controllers no pueden tocar el DbContext directo, convenciones de naming, manejo estándar de errores. Esto no es documentación que alguien puede ignorar — es lo que los gates van a hacer cumplir automáticamente."

specs/products-spec.md

"La feature en sí: la entidad Product, los 5 endpoints REST, las reglas de negocio — precio mayor a cero, nombre único, stock no negativo — y los edge cases. Fíjense que no menciona una sola clase, ni carpeta, ni ORM. Es el QUÉ, no el CÓMO — esa es la regla de SDD: la spec no prescribe implementación."

specs/verification.md

"Acá está la trazabilidad: cada regla de arriba baja a un criterio chequeable con un ID — AC-01, AC-02, ARCH-01... — y cada ID tiene un test con el mismo nombre en /gates. Si un test falla, se traza directo a la línea de este documento que lo originó."

gates/ — qué es un gate, específicamente

"Acá está el corazón del harness. Un 'gate' es un chequeo automático que decide si el código entra o no — como una tranquera: no importa quién escribió el código, humano o agente, si no pasa el gate, no entra. Tengo dos tipos:

1. **Gates funcionales** (`Gates.Api.Tests`) — 13 tests xUnit, uno por cada criterio AC-xx, que le pegan por HTTP a la API ya corriendo. Son *black-box*: no tienen ninguna referencia de compilación a `/src`, así que este proyecto compila aunque `/src` esté vacío.
2. **Gates de arquitectura** (`Gates.Architecture.Tests`) — acá está lo interesante. Uso una librería que se llama NetArchTest, que no lee el código fuente, **lee el ensamblado ya compilado** — el binario — y verifica relaciones de dependencia reales entre las capas. Por ejemplo: 'ningún tipo del assembly Api puede depender de Microsoft.EntityFrameworkCore'. Esto es más fuerte que un linter de estilo, porque un linter mira sintaxis; esto mira si el compilador realmente generó una referencia a ese ensamblado."

scripts/verify.ps1

"Es el loop de feedback: compila los gates, si ya existe `src/Api` levanta la API, corre los 18 tests, y devuelve un exit code — cero si todo pasó, distinto de cero si algo falló. Rojo o verde, sin ambigüedad. Esto es lo que orquesta todo, y es lo mismo que le voy a pedir al agente que corra."

src/

"Vacío a propósito, solo un README que dice que no hay que implementar todavía. Punto clave: **todo en /gates ya compila y corre aunque /src esté vacío**. El harness no necesita que la app exista para poder evaluarla — 'no implementado' es un estado tan detectable como 'arquitectura violada'."

4. Demo en vivo (~7 min)

Paso A — mostrar el rojo (1 min)

[EN VIVO: `.\scripts\verify.ps1`]

"Build de gates OK, pero los 18 tests fallan: los 13 funcionales porque no hay API corriendo, los 5 de arquitectura porque no encuentran los ensamblados de `/src`. Esto no es un error del harness — es el harness funcionando. Detecta 'no implementado'."

Paso B — pedirle a Copilot que implemente (~5 min)

[EN VIVO: pegar el prompt en Copilot Chat, modo Agent]

"Le pido a Copilot, en modo Agent, que implemente `/src` siguiendo la constitution y la spec, sin tocar `/gates`, y que corra `verify.ps1` iterando hasta que dé verde. Mientras corre esto, este es el loop de feedback en acción: el agente no me pregunta a mí si está bien, le pregunta al gate."

[Esperar a que termine, correr `verify.ps1` de nuevo si hace falta confirmar]

"18 de 18 en verde. El agente generó la implementación completa — cuatro proyectos respetando las capas — y el gate la aceptó sin que yo revisara una sola línea a mano."

Paso C — inyectar la violación y cazarla (~2 min)

"Ahora el momento que más me interesa mostrarles: ¿qué pasa si alguien — un agente o una persona — viola la arquitectura?"

[EN VIVO: editar `ProductsController.cs`, agregar un campo `DbContext` + `using Microsoft.EntityFrameworkCore;` (y el `PackageReference` en `Api.csproj` si hace falta para que compile)]

[EN VIVO: `.\scripts\verify.ps1`]

"ARCH01 falla, y no de forma genérica — señala exactamente el tipo ofensor: `Api.Controllers.ProductsController`. El resto sigue verde. Esto es la prueba de que el gate entiende arquitectura real, no solo sintaxis."

[Revertir el cambio, correr `verify.ps1` una vez más para cerrar en verde]

5. Cierre (~1 min)

"El mensaje central: el spec fue el feedforward, el gate fue el feedback, y ninguno de los dos requirió que alguien leyera el diff línea por línea. Elegí un caso chico y Greenfield a propósito, para poder mostrar el ciclo completo en vez de un pedazo grande a medio terminar. Esta es mi interpretación de Harness — seguramente distinta a la de cada uno de ustedes, y de eso se trata esta semana."

Por si preguntan algo (respuestas cortas)

- **¿Y si el agente edita los gates para que pasen?** — Es un gap real de esta demo mínima: el guardrail acá es el prompt ("no toques `/gates`") más revisión humana del diff. En una versión productiva (tipo 8090), esto se resuelve con permisos: el agente literalmente no tiene acceso de escritura a `/gates`.
- **¿Por qué NetArchTest y no solo un linter?** — Un linter mira sintaxis/estilo; NetArchTest inspecciona el ensamblado compilado y verifica dependencias reales entre capas.
- **¿Esto es específico de .NET?** — El patrón (specs + gates + loop de feedback) es agnóstico de stack. En otros lenguajes el equivalente sería ArchUnit (Java), import-linter (Python), dependency-cruiser (JS/TS).
- **¿Por qué Greenfield y no Brownfield?** — Elección deliberada de scope para esta semana: valida el mecanismo completo en el tiempo disponible. Adaptar un Brownfield a SDD es un paso lógico siguiente, no algo que descarté por dificultad técnica.